

Microservices_Basics Протокол

Github: https://github.com/Ok-ss/Microservices/tree/micro_basics

Короткий опис системи:

Сервіс *facade* реалізує логіку банківської системи. Із *POST* запитом клієнт відправляє повідомлення у вигляді *user_id* і суму, яку потрібно додати чи відняти. Сервіс *facade* викликає *logging* сервіс, щоб записати транзакцію, і сервіс *counter*, щоб оновити рахунок. Запит *GET* дозволяє отримати дані про баланс і транзакції одного користувача, або дані про баланси всіх користувачів.

Прийняті рішення:

Для реалізації цього завдання я прийняла рішення використовувати *python FastApi*, через його простоту у використанні та достатню ефективність. Потенційно, я могла б досягнути більшої швидкості, використовуючи компільовані мови програмування, але для цього потрібно було б імплементувати надто багато деталей мережевого зв'язку. Це могло б призвести до більш крихкої системи, особливо враховуючи, що майбутні завдання будуть надбудовуватися над цим. Продуктивності, яку дає *python*, на мою думку, цілком достатньо, щоб вважати цю систему ефективною, тому я вирішила не ускладнювати завдання марно.

Усі функції сервісу *facade* є асинхронними, а функції інших сервісів асинхронно викликаються, тому система досягає максимальної паралелізації, можливої в цих умовах. Сервіс *facade* робить запити до сервісів *logging* та *counter* одночасно і, отримавши від них відповіді, повертає користувачу очікуване значення. Для всіх запитів обрано найбільш ефективний метод роботи, покладаючись на особливості реалізації мови програмування і поставлене завдання.

Час відправки запиту використовується в якості *transaction_id*, для зменшення кількості даних, які необхідно зберігати в базі. *Logging service* використовує його як ключ, але додає його і в словник транзакції, який є значенням, тому що ці дані можуть бути корисними в майбутньому і не погіршують значно роботу сервісів.

Також, для майбутніх покращень я реалізувала можливість отримати загальний час запитів *POST* і *GET* на *logging service* та на *counter service* за допомогою констант (*log_time_total*, *transactions_time_total - logging service*, відповідно, *counter_time_total*, *balances_time_total - counter service*, відповідно). Крім того я реалізувала також і кінцеві точки для сервісу *facade*, щоб отримати і занулити ці метрики.

Команди та відповіді:

Підвищення балансу користувача user1 на 1:

```
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d '{"user_id":"user1", "amount":1}'
{"transaction_id":"1771768382.0249472","balance":1}schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$
```

Для кращої видимості, ось текст (C - command, R - response):

```
C: curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d
'{"user_id":"user1", "amount":1}'
R:{"transaction_id":"1771768382.0249472","balance":1}
```

Отримання нового балансу користувача user1:

```
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl http://localhost:8000/user/user1
{"balance":1,"transactions":[{"timestamp":"1771768382.0249472","user_id":"user1","amount":1}]schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$
```

```
C: curl http://localhost:8000/user/user1
R:{"balance":1,"transactions":[{"timestamp":"1771768382.0249472","user_id":"user1","amount":1}]}
```

Підвищення балансу користувача user1 ще на 1 і демонстрація роботи запиту GET з кількома транзакціями:

```
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d '{"user_id":"user1", "amount":1}'
{"transaction_id":"1771768503.4740927","balance":2}schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl http://localhost:8000/user/user1
{"balance":2,"transactions":[{"timestamp":"1771768498.6436107","user_id":"user1","amount":1},{"timestamp":"1771768503.4740927","user_id":"user1","amount":1}]schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$
```

```
C: curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d
'{"user_id":"user1", "amount":1}'
R:{"transaction_id":"1771768503.4740927","balance":2}

C: curl http://localhost:8000/user/user1
R:{"balance":2,"transactions":[{"timestamp":"1771768498.6436107","user_id":"user1","amount":1},{"timestamp":"1771768503.4740927","user_id":"user1","amount":1}]}
```

Введення нового користувача user2 і демонстрація роботи запиту GET для отримання балансів усіх користувачів

```
amount":1}]schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d '{"user_id":"user2", "amount":4}'
{"transaction_id":"1771768551.0099218","balance":4}schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl http://localhost:8000/accounts
{"user1":2,"user2":4}schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$
```

```
C: curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d
'{"user_id":"user2", "amount":4}'
R:{"transaction_id":"1771768551.0099218","balance":4}

C: curl http://localhost:8000/accounts
R:{"user1":2,"user2":4}
```

Демонстрація зменшення балансу:

```
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d '{"user_id":"user2", "amount":-2}'
```

```
C: curl -X POST http://localhost:8000/send -H "Content-Type: application/json" -d '{"user_id":"user2", "amount":-2}'
R:{"transaction_id":"1771784941.7678213","balance":2}
```

Випробування за часом:

Для випадку з 10 різними потоками, кожен з яких поповнює власний рахунок загальний час складає 101.42373037338257 секунди, а кількість оброблених запитів за секунду - 98.59625516815322. Отже, навіть для 100 користувачів, що одночасно надсилають запити на свої акаунти, не виникне жодних проблем з їх обробкою. Також, баланс кожного користувача - 1000, як і очікувалось

```
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/post10.py
Total time: 101.42373037338257
Requests/sec: 98.59625516815322
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl http://localhost:8000/accounts
{"user0":1000,"user4":1000,"user1":1000,"user2":1000,"user3":1000,"user7":1000,"user6":1000,"user5":1000,"user8":1000,"user9":1000}
```

Для випадку, коли 10 різних користувачів всі створюють запити для 1 *user_id*, час не збільшується, попри те що потрібно утримати точність підрахунку. Загальний час стає 100.80150866508484 секунди, а к-сть запитів, що обробляються за секунду складає 99.20486441552391. Результат також відповідає очікуванням

```
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/post1.py
Total time: 100.80150866508484
Requests/sec: 99.20486441552391
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ curl http://localhost:8000/accounts
{"user0":10000}
```

Додатковий аналіз з розподілом на POST і GET:

Я також вирішила провести загальний аналіз того, скільки часу займають запити *POST* та *GET* окремо. Для цього я змінила файли з попередніх тестів, щоб вони надсилали саме *GET*, або різні види запитів.

Що цікаво, час для запитів *GET* для конкретного користувача (*/user/user_id*) відрізняється вдвічі. Така ситуація виникає через те, що в другому випадку потрібно повертати стрічку з транзакціями, яка прямує до 10 000, а в першому - лише до 1 000

```
schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/get_user10.py
Total time: 489.29691982269287
Requests/sec: 20.437488148553463

schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/get_user_1.py
Total time: 856.5872631072998
Requests/sec: 11.674233823795904
```

У випадку з запитамі для всіх користувачів (*/accounts*) ситуація з часом знову майдже не змінюється, бо різниця між кількістю символів у стрічці не така велика, як у

попередньому порівнянні.

```
Requests/sec: 92.98338778828888
• schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/get_accs10.py
Total time: 85.62737989425659
Requests/sec: 116.78507519848502

• schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/get_accs1.py
Total time: 84.76918745040894
Requests/sec: 117.96739240717777
```

Як і очікувалось, різниця між часом виконання скриптів, які задають випадкові запити, досить велика через час, який займає *GET* для одного конкретного користувача.

```
amount": -2}}schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/random10.py
Total time: 104.16698837280273
Requests/sec: 95.99970351654065

• schuch@Schuch-vostro-1510:~/Desktop/UCU/APZ/Lab1/Microservices$ python3 time_testing/random1.py
Total time: 220.71790051460266
Requests/sec: 45.30670134449925
```

При цьому всі скрипти не створюють помилок у балансах і валідно зберігають всі транзакції.

Висновок:

Отже, мною була імплементована основа для банківської системи, що складається з сервісів *facade*, *counter* і *logging*. Кожен з них коректно виконує свою функцію і ефективно співпрацює з двома іншими для створення суцільної системи. Для роботи було використано *python FastApi* через його простоту і достатню ефективність.

Продуктивність забезпечується асинхронністю та використанням особливостей середовища і може бути підрахована за допомогою скриптів у директорії *time_testing* та кінцевих точок */metrics* та */metrics/reset*. В результаті система працює добре для сценаріїв з 10-ма користувачами незалежно від того, на який акаунт вони роблять запити. Найбільш часозатратним може стати запит *GET* для одного користувача, але лише на етапі, коли у користувача досить багато транзакцій.